

Assignment – 1: IAS processor design

Team: -

1. Kotyada Parthiv (IMT2023559)
2. Taral Sri Sai Ram (IMT2023588)
3. Dheeraj Muppiri (IMT2023596)

Program- Factorial Calculation

1. Introduction

In our project, we developed an assembly-level language optimized for our IAS processor. The machine code representation consists of two 40-bit numbers, each divided into two 20-bit blocks, known as Left Instruction and Right Instruction. The Left instruction has 8 bits for the opcode and 12 bits for the address, followed by the same settings in the Right Instruction. Expanding beyond the 21 instructions in the IAS framework, we introduced 2 new instructions: the COMPARE and the DEC. The COMPARE instruction makes the program run if $n > 0$, while the DEC instruction facilitates reducing the Accumulator (AC) value by one. In addition, we added a HALT instruction to end the execution of the processor which is responsible for converting the assembly code into this machine code, enabling the processor to process instructions and deliver the desired result.

2. C & Assembly Code

a. Here is the the C version of the program that we wish to implement.

```
1  #include <stdio.h>
2
3  int fact(int n)
4  {
5      static int a[100];
6      static int c = 0;
7
8      if (n < 0)
9      {
10         return 1;
11     }
12
13     if (n < c)
14     {
15         return a[n];
16     }
17
18     int result = (n == 0) ? 1 : n * fact(n - 1);
19     a[n] = result;
20     c = n;
21
22     return result;
23 }
24
25 int main()
26 {
27     int num;
28
29     while (1)
30     {
31         printf("Enter a number: ");
32         scanf("%d", &num);
33
34         if (num < 0)
35         {
36             break;
37         }
38
39         int factorial = fact(num);
40         printf("%d! = %d\n", num, factorial);
41     }
42
43     return 0;
44 }
```

b. The IAS assembly program for our C program.

```
13
COMPARE M(1)    JUMP+ M(7,0:19)
LOAD MQ, M(8)   LOAD M(1)
MUL M(1)        STOR M(8)
LOAD M(1)       DEC
STOR M(1)        JUMP+ M(3,0:19)
NOP            HALT
1
```

The opcodes we assigned are:

COMPARE- 10101010

DEC- 11111111

3. Assembler

We wrote an assembler in Python that can read in the above-written IAS assembly program for Factorial Calculation.

```
1 i_c= [['13\n'], ['COMPARE M(0)', 'JUMP+ M(6,0:19)\n'], ['LOAD MQ, M(7)', 'LOAD M(0)\n'], ['MUL M(0)', 'STOR M(7)\n'], ['LOAD M(0)', 'DEC\n'], ['STOR M(0)', 'JUMP+ M(2,0:19)\n'], ['NOP', 'HALT\n'], ['1\n']]
2 j = []
3 def best(n):
4     b_r = bin(n)[2:]
5     b_p = '0' * (40 - len(b_r))
6     r_b = b_p + b_r
7     return r_b
8 def opcode(s):
9     if s == 'DEC\n':
10         return '11111111'
11     elif s == 'LOAD MQ, M()\n':
12         return '00001001'
13     elif s == 'STOR M()\n':
14         return '00100001'
15     elif s == 'LOAD M()\n':
16         return '00000001'
17     elif s == 'JUMP+ M(,.)\n':
18         return '00001111'
19     elif s == 'MUL M()\n':
20         return '10001011'
21     elif s == 'NOP\n':
22         return '00000000'
23     elif s == 'DEC':
24         return '11111111'
25     elif s == 'LOAD MQ, M()':
26         return '00001001'
27     elif s == 'STOR M()':
28         return '00100001'
29     elif s == 'LOAD M()':
30         return '00000001'
```

```

31     elif s == 'JUMP+ M(,:)':
32         return '00001111'
33     elif s == 'MUL M()':
34         return '10001011'
35     elif s == 'NOP':
36         return '00000000'
37     elif s == 'HALT':
38         return '10101010'
39     elif s == 'HALT\n':
40         return '10101010'
41     elif s == 'COMPARE M()':
42         return '11110000'
43     elif s == 'COMPARE M()\n':
44         return '11110000'
45     s = []
46     def bry(n):
47         s = ''
48         for i in range(12):
49             x = n % 2
50             n = n // 2
51             s = str(x) + s
52         return s
53     def r_s(i_s):
54         r_st = ''.join(char for char in i_s if not char.isdigit())
55         return r_st
56     def r_s1(i_s):
57         r_st = ''.join(char for char in i_s if char.isdigit())
58         return r_st

```

```

59     for i in i_c:
60         if len(i) == 1:
61             j.append(r_s1(i[0]))
62         elif len(i) == 2:
63             for k in i:
64                 j.append(r_s1(k))
65     j.remove("")
66     j.remove("")
67     j.remove("")
68     j[2] = "7"
69     j[9] = "3"
70     for delta in i_c:
71         if len(delta) == 1:
72             s.append(best(int(delta[0].strip())))
73         elif len(delta) == 2:
74             for alpha in delta:
75                 alpha = r_s(alpha)
76                 alpha.replace('\n', "")
77                 s.append(opcode(alpha))
78                 if alpha != 'DEC' and alpha != 'NOP':
79                     a = j[0]
80                     a = int(a)
81                     s.append(bry(a))
82                     a = str(a)
83                     j.remove(a)
84             else:
85                 s.append('000000000000')

```

```

86     i_l = s
87     o_l = []
88     i_l[19] = "00001111"
89     i = 0
90     while i < len(i_l):
91         c_e = i_l[i]
92         c_e_l = len(c_e)
93         if c_e_l == 8:
94             s_v = c_e + i_l[i + 1] + i_l[i + 2] + i_l[i + 3]
95             o_l.append(s_v)
96             i += 4
97         else:
98             o_l.append(c_e)
99             i += 1
100     for element in o_l:
101         print(element)

```

Output:

[illegible]

4. IAS Processor

```

1  #global ac,ir,ibr,mar,mbr,mq,pc
2  pc = 1
3  ir = ''
4  t = 0 # toggle terminate
5  mq = 0
6  def fetch():
7      global pc,mar,mbr, memory
8      mar = pc
9      mbr = memory[mar]
10     pc = pc+1
11     mq=0
12
13
14 def decode():
15     global ir,mar,ibr,mbr,t
16     ir = mbr[0:8]
17     mar = int(mbr[8:20],2)
18     ibr = mbr[20:40]
19     execute(ir,mar)
20     ir=ibr[0:8]
21     mar=int(ibr[8:20],2)
22     mq=0
23     execute(ir,mar)
24
25 def execute(ir,mar):
26     global mbr, memory, mbr, mq, pc, ac
27     if ir == '00001001': #LOAD MQ, M()
28         mbr = memory[mar]
29         mq = int(mbr,2)
30     elif ir == '00100001': #STOR M()
31         mbr = bin(ac)[2:].zfill(40)
32         memory[mar] = mbr
33     elif ir == '00000001': #LOAD M()
34         mbr = memory[mar]
35         ac = int(mbr,2)
36     elif ir == '10001011': #MUL M()
37         mbr = memory[mar]
38         ac = mq * int(mbr,2)
39     elif ir == '11111111': #DEC
40         ac = ac - 1
41     elif ir == '00001111': #JUMP+ M(,:)
42         if ac>0:
43             pc = mar
44     elif ir == '10101010': #COMPARE M()
45         if int(memory[mar],2)>0:
46             ac = -1
47         else:
48             ac = 1
49     elif ir == '00000000': #NOP
50         pass
51     elif ir == '10000000': #HALT
52         print("the end")

```

```

53 print(f"mar = {bin(mar)[2:].zfill(12)}\n mbr = {mbr}\n ibr = {ibr}\n ir = {ir}\n ac = {bin(ac)[2:].zfill(40)}\n mq = {bin(mq)[2:].zfill(40)}")
54 memory=[]
55 with open('binaryCode.txt','r') as f:
56     for line in f:
57         memory.append(line.strip())
58     print(memory)
59
60 while(ir != '10000000'):
61     fetch()
62     decode()
63     #print(f"mar = {bin(mar)[2:].zfill(12)}\n mbr = {mbr}\n ibr = {ibr}\n ir = {ir}\n ac = {bin(ac)[2:].zfill(40)}\n mq = {bin(mq)[2:].zfill(40)}")
64 print(f" result = {int(memory[7],2)}")

```

Output:

[illegible]